

Roll Number: _____

Name: _____

Q1	Q2	Q3	Q4	Q5	Total

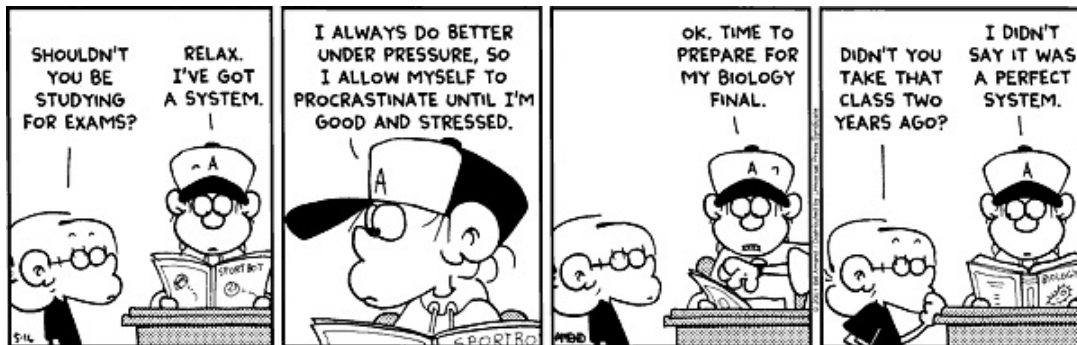
CS339: Abstractions and Paradigms for Programming

Midsem Exam (2 hours)

September 15th, 2025

Instructions:

- Write neat, clear and crisp answers in the space provided.
- There are separate spaces provided for rough work; please don't write answers in that space.
- In case you make an assumption, write it down before the corresponding answer.
- Each Q carries 6 marks.



Q1. Consider the following Scheme code.

```
(define (make-counter n)
  (lambda (m)
    (cond ((eq? m 'inc) (begin (set! n (+ n 1)) n))
          ((eq? m 'dec) (begin (set! n (- n 1)) n))
          (else (error "unknown message")))))

(define C1 (make-counter 5))
(define C2 C1)
(C2 'inc)
```

(a) Draw the environment diagram just before evaluating `(C2 'inc)`.

[3]

(b) What happens if we now call `(C1 'dec)`? Which variable is modified?

[1.5]

(c) Give a small code sequence such that the answer of (b) would differ under dynamic scoping. [1.5]

Q2. Suppose we want a Scheme function `exists?` that checks whether *any element* in a list satisfies a predicate:

```
> (exists? even? '(2 4 5 7))  
#t
```

(a) Define `exists?` using `foldr`. [2]

(b) Explain why might this implementation be inefficient. [1]

(c) Reimplement `exists?` without `foldr` such that the efficiency concern you pointed out in (b) is gone. [2]

(d) Any thoughts on how could one make the `foldr`-version efficient?

[1]

Q3. (a) In the class we had discussed two representations of complex numbers: Polar and Rectangular. Polar has attributes *radius* and *angle*, while Rectangular has attributes *real* and *imaginary*. Each representation makes certain operations easier (e.g., addition in rectangular, multiplication in polar). A natural idea is: why not store all four attributes together inside a complex number object so that every operation becomes easy. Can you find at least two problems with this approach? [2]

(b) Suppose we may support rational numbers later, which also can be added and multiplied. Propose a class/interface hierarchy for our number system, with insights on when to use when. [2]

(c) Suppose an integer field is 4 bytes, and that an object is just composed of storage for its fields¹. Draw a box representing the shapes of objects of the Java classes A and B defined below (label field `f` of class C as `C : f`), and indicate the total memory allocated when instantiating each. [2]

```
class A {  
    int x, y, z;  
}  
class B extends A {  
    int y, z;  
}
```

¹An object comprises of many other things too, but more about the *cool* things that they enable in CS6004.

Q4. Recall the definitions of Church numerals from Quiz 1 and those of booleans from the class:

$0 : \lambda f . \lambda x . x,$ $1 : \lambda f . \lambda x . f x,$ $2 : \lambda f . \lambda x . f f x,$ and so on...

$T : \lambda a . \lambda b . a,$ $F : \lambda a . \lambda b . b$

Now consider the following function Z that is intended to test whether a Church numeral is zero²:

$Z : \lambda n . n (\lambda x . F) T$

(a) Show that the above definition of Z works. That is, it returns T when applied to 0 but F when applied to other Church numerals (such as 1). [2]

²In the Scheme definition of Church numerals, we had defined an `is-zero?` function that relied on the existence of a `null?` function that does not exist in pure lambda calculus.

(b) Define a logical-negation term N such that $(N\ T)$ gives F and $(N\ F)$ gives T . [2]

(c) Can you find a term that is not β -reducible to our 0, but for which our definition of Z still returns T ? [2]

Q5. Repeat to yourself five times: “*Marks are just an illusion; they do not reflect what I am and they exist just for administrative reasons.*” Now suppose you are writing a Scheme program that represents a class gradebook as a list of pairs (*name . marks*). You would get a PC if your answer for all the questions is a one-liner body.

(a) Write a function `top-students` that takes such a list and returns a list of names of students with scores above 80. Use higher-order functions (`map`, `filter`). [2]

(b) Write a function `add-pc` that adds 5 marks to every student, but caps marks at 100. Use `map`. [2]

(c) Say we want to check if all students passed ($\text{marks} \geq 40$). Implement `all-passed?` using `fold(r/l)`. [2]

SPACE FOR YOUR MUSINGS

SPACE FOR YOUR MUSINGS